

Aria Land (NFT)

SMART CONTRACT

Security Audit

Performed on Contracts:

LandofAria.sol

MD5: 1737f0946e6e72e7c9b4cbf94bb38353

Inherited Libraries

Platform

ETH

hashlock.com.au

MAY 2023

Table of Contents

Executive Summary	4
Project Context	4
Roadmap	7
Aria Land Project Leadership	8
Audit scope	10
Security Rating	11
Standardised Checks	12
Intended Smart Contract Functions	14
Function List	15
Code Quality	20
Audit Resources	20
Dependencies	20
Severity Definitions	21
Audit Findings	21
Centralisation	30
Conclusion	31
Our Methodology	32
Disclaimers	34
About Hashlock	35
Appendix	36

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Aria Land team partnered with Hashlock to conduct a security audit of their ERC721 smart contract used for their NFT's prior to their project's launch. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that it is ready for mainnet deployment.

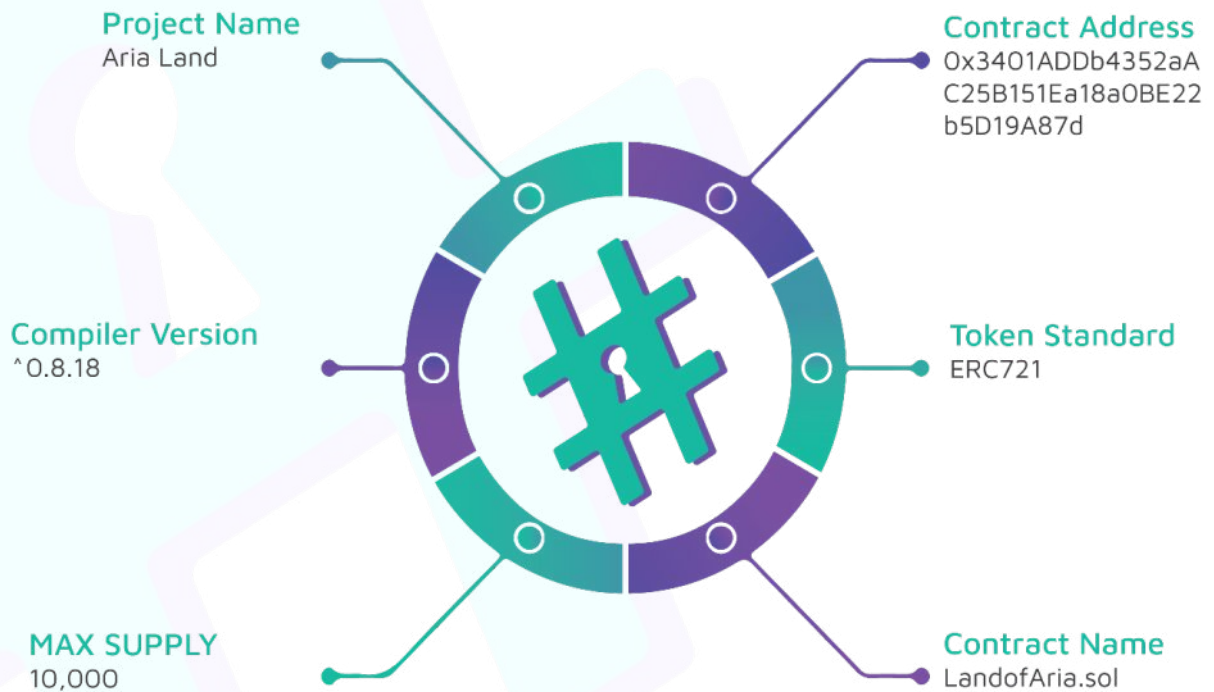
Project Context

Aria Land is an upcoming on-chain game project aiming to build an engaged community via an exciting roadmap. The Aria Land team engaged Hashlock to ensure their reward contract is ready for Launch.

Project Name: Aria Land

Contract Address: N/A

Compiler Version: ^0.8.18

Visuals:**Visualised Context:**

Project Artwork:



Gordite

Raffle Prize: **2.66 ETH**

Free to play tokens: **5**

Rare

Raffle Prize: **5.32 ETH**

Royalties: **5%**



Luminara

Raffle Prize: **3.99 ETH**

Free to play tokens: **5**

Rare

Raffle Prize: **5.32 ETH**

Royalties: **5%**



Ellor

Raffle Prize: **1.33 ETH**

Free to play tokens: **5**

Rare

Raffle Prize: **5.32 ETH**

Royalties: **5%**

Roadmap

PHASE 1

- LAUNCH OFFICIAL ARIA Land WEBSITE
- DISCORD OPENS TO PUBLIC
- LAUNCH MARKETING CAMPAIGN
- GAME 01 COMMENCES

PHASE 2

- GAME 01 WINNERS ANNOUNCED
- GAME 02 COMMENCES
- RELEASE 1st NFT GIVEAWAY
- OFFICIAL WHITE-LIST RAFFLE COMMENCES
- DEVELOPMENT OF BETA VERSION ARIA LAND

PHASE 3

- RELEASE OF PUBLIC SALE
- AIRDROP \$FSHK TOKEN TO NFT HOLDERS
- ARIA RAFFLE COMMENCES
- ARIA Metaverse WHITE PAPER RELEASE

PHASE 4

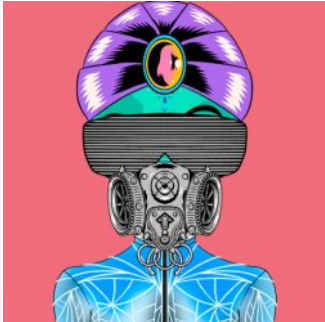
- GAME 03 COMMENCES
- ARIA Metaverse DEVELOPMENT

PHASE 5

- GAME 03 WINNERS ANNOUNCED
- MORE PERKS COMING SOON

Aria Land Project Leadership

Title	Name	Photo	Links
CEO	TheFreakNetwork		Instagram Twitter Discord
COO	TheGreekNetwork		Instagram Twitter Discord
Artist	TheARTNetwork		Instagram Twitter Discord
Designer	The3DNetwork		Instagram Twitter Discord

Marketing	CryptoNick		Instagram Twitter Discord
Designer	MetaGirl		Instagram Twitter Discord

Audit scope

We at Hashlock audited the solidity code within the Aria Land Project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken through both manual and software assisted analysis.

Description	Project Review and Security Analysis Report for Aria Land Protocol Smart Contracts and other factors.
Platform	Ethereum / Solidity
Audit Date	April, 2023
Contract 1	LandOfAria.sol
Contract 1 MD5 Hash	1737f0946e6e72e7c9b4cbf94bb38353

Security Rating

After our audit and analysis, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified some critical and other vulnerabilities that have mostly been addressed.



Not Secure	Vulnerable	Secure	Hashlocked
------------	------------	--------	------------

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

All vulnerabilities initially identified are yet to be resolved.

Hashlock found:

0 Critical severity vulnerabilities

0 High severity vulnerabilities

2 Medium severity vulnerabilities

15 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Passed
	Outdated Solidity Version	Passed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Passed
	Lack of check on input parameters	Reviewed
	Function input parameters check bypass	Passed
	Correct Access control	Passed
	Built in emergency features	Passed
	Correct event logs	Reviewed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Reviewed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Reviewed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed

	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Passed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Initial Audit Result: FAILED

Revised Audit Result: PASSED



Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
LandOfAria.sol - Max supply 10,000 - Uses merkle tree for whitelisting - Owners can change presale, private, and public minting prices. - Owners can create airdrops	Contract achieves this functionality.

Function List

Aria Land

- From LandofAria
- From Pausable
- From Context
- From Ownable
- From ReentrancyGuard
- From ERC721Enumerable
- From ERC721

Functions

	Functions	Visibility	Observation	Conclusion
1	constructor(string) - From LandofAria	Public	Passed	No Issue
2	airdrop(address[],uint256[]) - From LandofAria	External	access only Owner	No Issue
4	buildBlindMetadata(uint256) - From LandofAria	Public	Passed	No Issue
7	disableSale() - From LandofAria	External	access only Owner	No Issue
8	enableFfSale() - From LandofAria	External	access only Owner	No Issue
9	enablePublicSale() - From LandofAria	External	access only Owner	No Issue
10	enableWhitelistSale() - From LandofAria	External	access only Owner	No Issue
11	isFF(bytes32[],address) - From LandofAria	Public	Passed	No Issue
12	isWhitelisted(bytes32[],address) - From LandofAria	Public	Passed	No Issue
13	mint(uint256,bytes32[]) - From LandofAria	Public	Overpay Vulnerability	Refer Audit Findings
14	pause() - From LandofAria	External	access only Owner	No Issue

15	safeTransferMany(address,addresses[],uint256[]) - From LandofAria	External	access only Owner	No Issue
16	setMerkleRoot(bytes32,bytes32) - From LandofAria	External	access only Owner	No Issue
17	setPrices(uint256,uint256,uint256) - From LandofAria	External	access only Owner	No Issue
18	setPurchaseLimits(uint256,uint256) - From LandofAria	External	access only Owner	No Issue
19	setURI(string) - From LandofAria	External	Reveal Bool Vulnerability	Refer Audit Findings
20	tokenURI(uint256) - From LandofAria	Public	Passed	No Issue
21	unpause() - From LandofAria	External	access only Owner	No Issue
22	withdraw(address,uint256) - From LandofAria	External	access only Owner	No Issue
23	_pause() - From Pausable	Internal	Passed	No Issue
24	_requireNotPaused() - From Pausable	Internal	Passed	No Issue
25	_requirePaused() - From Pausable	Internal	Passed	No Issue
26	_unpause() - From Pausable	Internal	Passed	No Issue
27	constructor() - From Pausable	Internal	Passed	No Issue
28	_msgData() - From Context	Internal	Passed	No Issue
29	_msgSender() - From Context	Internal	Passed	No Issue
30	_checkOwner() - From Ownable	Internal	Passed	No Issue
31	_transferOwnership(address) - From Ownable	Internal	Passed	No Issue
32	owner() - From Ownable	Public	Passed	No Issue
33	renounceOwnership() - From Ownable	Public	access only Owner	No Issue
34	transferOwnership(address) - From Ownable	Public	access only Owner	No Issue

35	<code>_nonReentrantAfter()</code> - From <code>ReentrancyGuard</code>	Private	Passed	No Issue
36	<code>_nonReentrantBefore()</code> - From <code>ReentrancyGuard</code>	Private	Passed	No Issue
37	<code>_addTokenToAllTokensEnumeration(uint256)</code> - From <code>ERC721Enumerable</code>	Private	Passed	No Issue
38	<code>_addTokenToOwnerEnumeration(address,uint256)</code> - From <code>ERC721Enumerable</code>	Private	Passed	No Issue
39	<code>_beforeTokenTransfer(address,address,uint256,uint256)</code> - From <code>ERC721Enumerable</code>	Internal	Passed	No Issue
40	<code>_removeTokenFromAllTokensEnumeration(uint256)</code> - From <code>ERC721Enumerable</code>	Private	Passed	No Issue
41	<code>_removeTokenFromOwnerEnumeration(address,uint256)</code> - From <code>ERC721Enumerable</code>	Private	Passed	No Issue
42	<code>supportsInterface(bytes4)</code> - From <code>ERC721Enumerable</code>	Public	Passed	No Issue
43	<code>tokenByIndex(uint256)</code> - From <code>ERC721Enumerable</code>	Public	Passed	No Issue
44	<code>tokenOfOwnerByIndex(address,uint256)</code> - From <code>ERC721Enumerable</code>	Public	Passed	No Issue
45	<code>totalSupply()</code> - From <code>ERC721Enumerable</code>	Public	Passed	No Issue
46	<code>__unsafe_increaseBalance(address,uint256)</code> - From <code>ERC721</code>	Internal	Passed	No Issue
47	<code>_afterTokenTransfer(address,address,uint256,uint256)</code> - From <code>ERC721</code>	Internal	Passed	No Issue
48	<code>_approve(address,uint256)</code> - From <code>ERC721</code>	Internal	Passed	No Issue
49	<code>_baseURI()</code> - From <code>ERC721</code>	Internal	Passed	No Issue
50	<code>_burn(uint256)</code> - From <code>ERC721</code>	Internal	Passed	No Issue

51	_checkOnERC721Received(address,address,uint256,bytes) - From ERC721	Private	Passed	No Issue
52	_exists(uint256) - From ERC721	Internal	Passed	No Issue
53	_isApprovedOrOwner(address,uint256) - From ERC721	Internal	Passed	No Issue
54	_mint(address,uint256) - From ERC721	Internal	Passed	No Issue
55	_ownerOf(uint256) - From ERC721	Internal	Passed	No Issue
56	_requireMinted(uint256) - From ERC721	Internal	Passed	No Issue
57	_safeMint(address,uint256) - From ERC721	Internal	Passed	No Issue
58	_safeMint(address,uint256,bytes) - From ERC721	Internal	Passed	No Issue
59	_safeTransfer(address,address,uint256,bytes) - From ERC721	Internal	Passed	No Issue
60	_setApprovalForAll(address,address,bool) - From ERC721	Internal	Passed	No Issue
61	_transfer(address,address,uint256) - From ERC721	Internal	Passed	No Issue
62	approve(address,uint256) - From ERC721	Public	Passed	No Issue
63	balanceOf(address) - From ERC721	Public	Passed	No Issue
64	constructor(string,string) - From ERC721	Public	Passed	No Issue
65	getApproved(uint256) - From ERC721	Public	Passed	No Issue
66	isApprovedForAll(address,address) - From ERC721	Public	Passed	No Issue
67	name() - From ERC721	Public	Passed	No Issue
68	ownerOf(uint256) - From ERC721	Public	Passed	No Issue

69	safeTransferFrom(address,address,uint256) - From ERC721	Public	Passed	No Issue
70	safeTransferFrom(address,address,uint256,bytes) - From ERC721	Public	Passed	No Issue
71	setApprovalForAll(address,bool) - From ERC721	Public	Passed	No Issue
72	symbol() - From ERC721	Public	Passed	No Issue
73	transferFrom(address,address,uint256) - From ERC721	Public	Passed	No Issue

Code Quality

This audit scope involves the solidity smart contracts of the Aria Land project, as outlined in the Audit Scope section. All contracts, libraries and interfaces intend to follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

Most code is mostly well commented on and closely follows best practice nat-spec styling. Most comments are correctly aligned with code functionality.

Audit Resources

We were given the Aria Land Protocol's smart contract code in the form of a Github and testnet Link.

As mentioned above, code parts are mostly well commented. The logic is straightforward, and therefore it is somewhat easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
Critical	Critical vulnerabilities are of immediate concern, and allow hackers or malicious code to steal funds, take ownership of the contract, or other devastating actions.
High	High vulnerabilities are not attacked as easily, but still have significant implications on the protocol.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

Critical

High

Medium

1) User's may accidentally overpay and lose funds in the mint function

```
LandofAria.sol  
  
87         if (FF_NFT_PRICE * _numOfTokens > msg.value) {  
88             revert invalidAmount(_numOfTokens, msg.value);  
89         }
```

It is possible for a user purchasing an NFT to accidentally overpay the mint fee during mint if minting directly from the contract.

Recommendation: Consider modifying the check to check that the msg.value is exactly equal to the fee.

Status: Resolved

2) URI Reveal status can be reversed by the owner.

Once NFT's are revealed, they should not be able to be unrevealed.

Recommendation: Remove functionality allowing tokens to be unrevealed. Require that reveal bool is not already true when changing status.

Status: Resolved

Low / Informational

1) uint256 can't be less than zero, change to `_amount == 0`

```
LandofAria.sol

313     function withdraw(address _to, uint256 _amount) external onlyOwner {
314         if (_amount <= 0) {
315             revert mustBeGreaterThanZero();
316         }
    }
```

uint256 can't be less than zero

Recommendation: change to `'_amount == 0'`

Status: Resolved

2) Best Practice: Put `nonReentrant` before `whenNotPaused`

```
LandofAria.sol

72     function mint(
73         uint256 _numOfTokens,
74         bytes32[] memory _merkleProof
75     ) public payable whenNotPaused nonReentrant {
    }
```

It is recommended to place the non-reentrant modifier before other modifiers to ensure it is executed first.

Recommendation: change order of modifiers

Status: Resolved

3) You can't change the public NFT Price without updating the whitelist and FF price variables.

```
LandofAria.sol

231     function setPrices(
232         uint256 _FF_NFT_PRICE,
233         uint256 _WHITELIST_NFT_PRICE,
234         uint256 _PUBLIC_NFT_PRICE
235     ) external onlyOwner {
236         FF_NFT_PRICE = _FF_NFT_PRICE;
237         WHITELIST_NFT_PRICE = _WHITELIST_NFT_PRICE;
238         PUBLIC_NFT_PRICE = _PUBLIC_NFT_PRICE;
239     }
```

You can update the whitelist prices to be the same as what they were, but this still is gas expensive. There should at least be an individual function to update the public NFT price.

Recommendation: Have individual functions for each change

Status: Acknowledged

4) No zero check on mint function

```
LandofAria.sol

72     function mint(
73         uint256 _numOfTokens,
74         bytes32[] memory _merkleProof
75     ) public payable whenNotPaused nonReentrant {
```

If a user enters zero for `_numOfTokens` this will successfully execute the mint function even though no NFTs will be minted. Add a check to make sure the user has entered a valid amount.

Recommendation: `if(_numOfTokens == 0) revert...`

Status: Resolved

5) Can set mint function to external as it's not being called by the contract



```
LandofAria.sol

72     function mint(
73         uint256 _numOfTokens,
74         bytes32[] memory _merkleProof
75     ) public payable whenNotPaused nonReentrant {
```

The mint function can be set to external to save gas.

Recommendation: change from public to external

Status: Resolved

6) uint256 default value is 0, so no need to reassign

```
LandofAria.sol

119         for (uint256 i = 0; i < _numOfTokens; i++) {
120             _safeMint(msg.sender, totalSupply() + 1);
121         }
```

Also change in the safeTransferMany, airdrop functions.

Recommendation: Default value is zero so no need to reinitialize.

Status: Resolved

7) Informational: disableSale and pause does the exact same thing

```
LandofAria.sol

282     function disableSale() external onlyOwner {
283         isFfSaleActive = false;
284         isPublicSaleActive = false;
285         isWhitelistSaleActive = false;
286     }
```

The whenNotPaused modifier is only used in the mint function and the disableSale function essentially stops minting therefore both do the same thing.

Recommendation: Only use one of the functions.

Status: Acknowledged

8) Some Parameters can be set to calldata instead of memory to save gas

```
LandofAria.sol

72     function mint(
73         uint256 _numOfTokens,
74         bytes32[] memory _merkleProof
75     ) public payable whenNotPaused nonReentrant {
```

The _merkleProof param can be set to calldata as it is a read only param. Other params can also be changed:

Recommendation: change from memory to calldata to save gas

Status: Resolved

9) Should check to ensure public sale is active in mint function, rather than relying on process of elimination

```
LandofAria.sol

105         } else {
106             if (PUBLIC_NFT_PRICE * _numOfTokens > msg.value) {
107                 revert invalidAmount(_numOfTokens, msg.value);
108             }
```

Recommendation: Check / require that public sale is active.

Status: Acknowledged

10) contract lacks custom events

Recommendation: Events can be helpful in tracking actions such as when each minting stage is active, prices are changed, purchase limits changed, etc..

Status: Acknowledged

11) Airdrop and safeTransferMany can be gas heavy in certain situations

Recommendation: Use caution while using these functions because they are gas heavy and may fail if the loop is too long.

Status: Acknowledged

12) Use SafeMath for multiplication

Recommendation: To avoid overflows and underflows and as best practice, utilise the SafeMath library for arithmetic operations

Status: Acknowledged

13) In the Airdrop() and setURIs() function there is no validation to check that the input arrays _to and _amount are non-empty.

Recommendation: As best practice and to avoid errors / mistakes, check that input arrays are not empty.

Status: Acknowledged

14) You can't change the public price without changing ff and presale prices.

Owner cannot change the NFT price separately. This means all prices need to be updated each time, even if presale is over. This is gas inefficient.

Recommendation: Add a function to separately change public price.

Status: Acknowledged

15) Testnet is deployed in one file rather than multiple files.

Currently all files are deployed together in the testnet address.

Recommendation: When launching on mainnet, all contracts and libraries should be in separate files under the same address.

Status: **Acknowledged**



Centralisation

The project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on private key security, and internal team responsibility.



Centralised

Decentralised



Conclusion

After Hashlocks analysis, the Aria Land NFT project seems to have a sound and well tested code base, now that our findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices.



Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

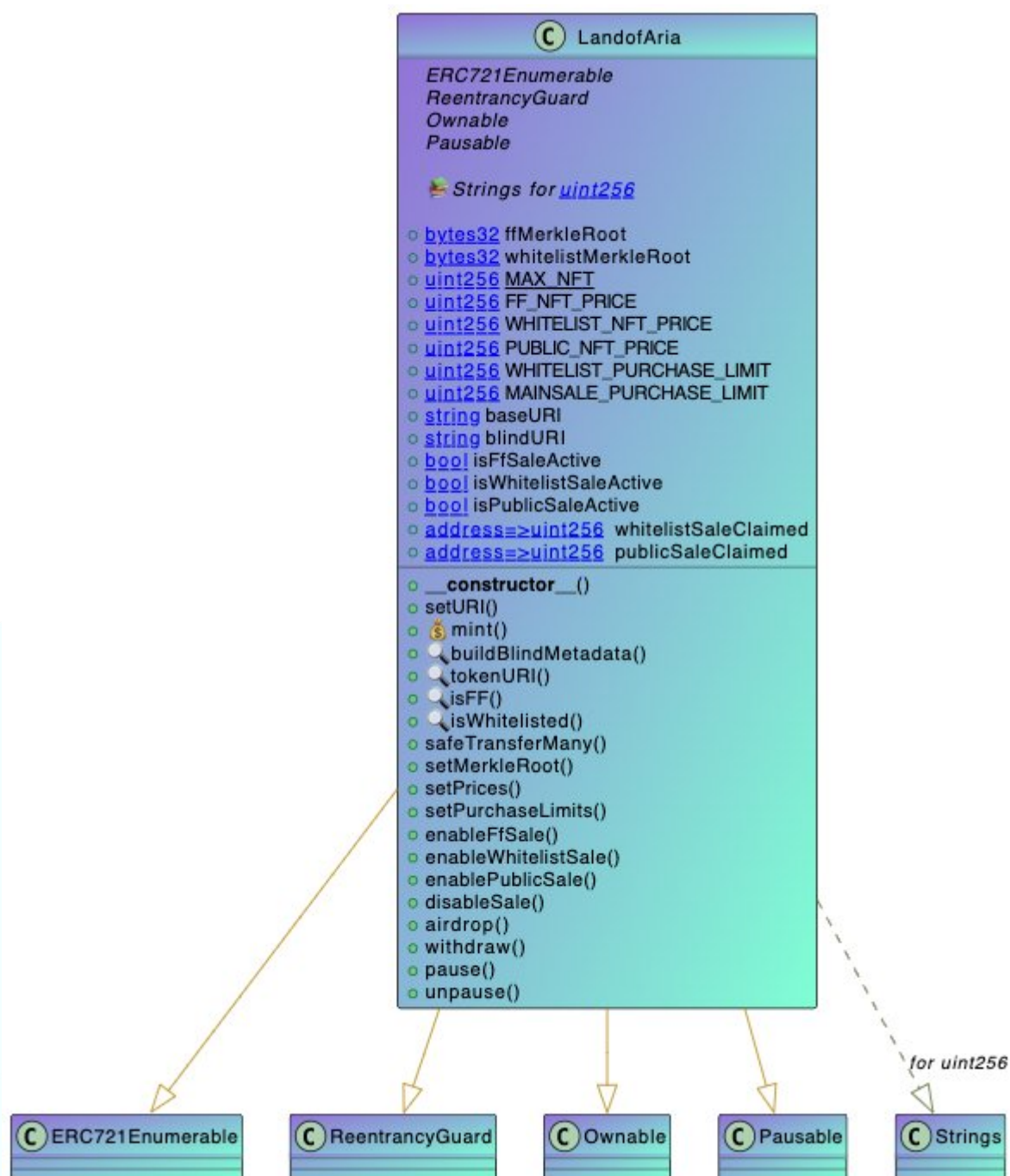
Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

Appendix

Code Flow Diagram



Slither Results Log

LandOfAria.sol

```
LandOfAria.withdraw(address,uint256).to (contracts/LandOfAria.sol#313) lacks a zero-check on :
- (success) = address(_to).call(value: _amount)() (contracts/LandOfAria.sol#322)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation

ERC721._checkOnERC721Received(address,address,uint256,bytes) (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#429-451) has external calls inside a loop:
IERC721Receiver(to).onERC721Received(_msgSender(),from,tokenId,data) (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#436-447)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#calls-inside-a-loop

Variable 'ERC721._checkOnERC721Received(address,address,uint256,bytes).retval (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#436)' in ERC721._checkOnERC721Received(address,address,uint256,bytes) (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#429-451) potentially used before declaration: retval == IERC721Receiver.onERC721Received.selector (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#437)
Variable 'ERC721._checkOnERC721Received(address,address,uint256,bytes).reason (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#438)' in ERC721._checkOnERC721Received(address,address,uint256,bytes) (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#429-451) potentially used before declaration: reason.length == 0 (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#439)
Variable 'ERC721._checkOnERC721Received(address,address,uint256,bytes).reason (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#438)' in ERC721._checkOnERC721Received(address,address,uint256,bytes) (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#429-451) potentially used before declaration: revert(uint256,uint256)(32 + reason,mload(uint256)(reason)) (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#444)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#pre-declaration-usage-of-local-variables

ERC721._checkOnERC721Received(address,address,uint256,bytes) (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#429-451) uses assembly
- INLINE ASM (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#443-445)
Address._revert(bytes,string) (node_modules/@openzeppelin/contracts/Utils/Address.sol#231-243) uses assembly
- INLINE ASM (node_modules/@openzeppelin/contracts/Utils/Address.sol#236-239)
Base64.encode(bytes) (node_modules/@openzeppelin/contracts/Utils/Base64.sol#20-91) uses assembly
- INLINE ASM (node_modules/@openzeppelin/contracts/Utils/Base64.sol#39-88)
Strings.toString(uint256) (node_modules/@openzeppelin/contracts/Utils/Strings.sol#18-38) uses assembly
- INLINE ASM (node_modules/@openzeppelin/contracts/Utils/Strings.sol#24-26)
- INLINE ASM (node_modules/@openzeppelin/contracts/Utils/Strings.sol#30-32)
MerkleProof.efficientHash(bytes32,bytes32) (node_modules/@openzeppelin/contracts/Utils/cryptography/MerkleProof.sol#215-222) uses assembly
- INLINE ASM (node_modules/@openzeppelin/contracts/Utils/cryptography/MerkleProof.sol#217-221)
Math.mulDiv(uint256,uint256,uint256) (node_modules/@openzeppelin/contracts/Utils/math/Math.sol#55-135) uses assembly
- INLINE ASM (node_modules/@openzeppelin/contracts/Utils/math/Math.sol#66-70)
- INLINE ASM (node_modules/@openzeppelin/contracts/Utils/math/Math.sol#86-93)
- INLINE ASM (node_modules/@openzeppelin/contracts/Utils/math/Math.sol#100-109)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage

Pragma version^0.8.18 (contracts/LandOfAria.sol#3) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6/0.8.16
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/access/Ownable.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/security/Pausable.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/security/ReentrancyGuard.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/token/ERC721/IERC721.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/token/ERC721/IERC721Receiver.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/token/ERC721/extensions/ERC721Enumerable.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/token/ERC721/extensions/IERC721Enumerable.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/token/ERC721/extensions/IERC721Metadata.sol#4) allows old versions
Pragma version^0.8.1 (node_modules/@openzeppelin/contracts/Utils/Address.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/Utils/Base64.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/Utils/Context.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/Utils/Strings.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/Utils/cryptography/MerkleProof.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/Utils/introspection/ERC165.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/Utils/introspection/IERC165.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/Utils/math/Math.sol#4) allows old versions
Pragma version^0.8.0 (node_modules/@openzeppelin/contracts/Utils/math/SafeMath.sol#4) allows old versions
solc-0.8.18 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in LandOfAria.withdraw(address,uint256) (contracts/LandOfAria.sol#313-327):
- (success) = address(_to).call(value: _amount)() (contracts/LandOfAria.sol#322)
Low level call in Address.sendValue(address,uint256) (node_modules/@openzeppelin/contracts/Utils/Address.sol#60-65):
- (success) = recipient.call(value: amount)() (node_modules/@openzeppelin/contracts/Utils/Address.sol#63)
Low level call in Address.functionCallWithValue(address,bytes,uint256,string) (node_modules/@openzeppelin/contracts/Utils/Address.sol#128-137):
- (success,returndata) = target.call(value: value)(data) (node_modules/@openzeppelin/contracts/Utils/Address.sol#135)
Low level call in Address.functionStaticCall(address,bytes,string) (node_modules/@openzeppelin/contracts/Utils/Address.sol#155-162):
- (success,returndata) = target.staticcall(data) (node_modules/@openzeppelin/contracts/Utils/Address.sol#160)
Low level call in Address.functionDelegateCall(address,bytes,string) (node_modules/@openzeppelin/contracts/Utils/Address.sol#180-187):
- (success,returndata) = target.delegatecall(data) (node_modules/@openzeppelin/contracts/Utils/Address.sol#185)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Parameter LandOfAria.setURI(string)._URI (contracts/LandOfAria.sol#65) is not in mixedCase
Parameter LandOfAria.mint(uint256,bytes32[])._numOfTokens (contracts/LandOfAria.sol#73) is not in mixedCase
Parameter LandOfAria.mint(uint256,bytes32[])._merkleProof (contracts/LandOfAria.sol#74) is not in mixedCase
Parameter LandOfAria.buildBlindMetadata(uint256)._tokenId (contracts/LandOfAria.sol#128) is not in mixedCase
Parameter LandOfAria.tokenURI(uint256)._tokenId (contracts/LandOfAria.sol#163) is not in mixedCase
Parameter LandOfAria.isFF(bytes32[],address)._merkleProof (contracts/LandOfAria.sol#179) is not in mixedCase
Parameter LandOfAria.isWhitelisted(bytes32[],address)._merkleProof (contracts/LandOfAria.sol#191) is not in mixedCase
Parameter LandOfAria.safeTransferMany(address,address[],uint256[])._to (contracts/LandOfAria.sol#204) is not in mixedCase
Parameter LandOfAria.safeTransferMany(address,address[],uint256[])._tokenIds (contracts/LandOfAria.sol#205) is not in mixedCase
Parameter LandOfAria.setMerkleRoot(bytes32,bytes32)._ffMerkleRoot (contracts/LandOfAria.sol#220) is not in mixedCase
Parameter LandOfAria.setMerkleRoot(bytes32,bytes32)._whitelistMerkleRoot (contracts/LandOfAria.sol#221) is not in mixedCase
Parameter LandOfAria.setPrices(uint256,uint256,uint256)._FF_NFT_PRICE (contracts/LandOfAria.sol#232) is not in mixedCase
Parameter LandOfAria.setPrices(uint256,uint256,uint256)._WHITELIST_NFT_PRICE (contracts/LandOfAria.sol#233) is not in mixedCase
Parameter LandOfAria.setPrices(uint256,uint256,uint256)._PUBLIC_NFT_PRICE (contracts/LandOfAria.sol#234) is not in mixedCase
Parameter LandOfAria.setPurchaseLimits(uint256,uint256)._WHITELIST_PURCHASE_LIMIT (contracts/LandOfAria.sol#245) is not in mixedCase
Parameter LandOfAria.setPurchaseLimits(uint256,uint256)._MAINSALE_PURCHASE_LIMIT (contracts/LandOfAria.sol#246) is not in mixedCase
Parameter LandOfAria.airdrop(address[],uint256[])._to (contracts/LandOfAria.sol#292) is not in mixedCase
Parameter LandOfAria.airdrop(address[],uint256[])._amount (contracts/LandOfAria.sol#293) is not in mixedCase
Parameter LandOfAria.withdraw(address,uint256)._to (contracts/LandOfAria.sol#313) is not in mixedCase
Parameter LandOfAria.withdraw(address,uint256)._amount (contracts/LandOfAria.sol#313) is not in mixedCase
Variable LandOfAria.FF_NFT_PRICE (contracts/LandOfAria.sol#22) is not in mixedCase
Variable LandOfAria.WHITELIST_NFT_PRICE (contracts/LandOfAria.sol#23) is not in mixedCase
Variable LandOfAria.PUBLIC_NFT_PRICE (contracts/LandOfAria.sol#24) is not in mixedCase
Variable LandOfAria.WHITELIST_PURCHASE_LIMIT (contracts/LandOfAria.sol#26) is not in mixedCase
Variable LandOfAria.MAINSALE_PURCHASE_LIMIT (contracts/LandOfAria.sol#27) is not in mixedCase
Function ERC721._unsafeIncreaseBalance(address,uint256) (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#503-505) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions

LandOfAria.blindURI (contracts/LandOfAria.sol#30) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
contracts/LandOfAria.sol analyzed (19 contracts with 84 detectors), 129 result(s) found
```



#Hashlock.